

Product Requirement Document (PRD) — V1

Focused MVP

Bank Statement Transaction Analyzer

Based on the uploaded roadmap and architecture direction.

1. Product Overview

Product Name

Transaction Analyzer

Objective

Build a lightweight web application that allows users to upload a bank statement PDF, search transactions by a specific person/entity name, and instantly calculate:

- Total credited amount
- Total debited amount
- Transaction count
- Matching transaction list

The system should work entirely in-memory for V1 without database persistence.

2. Problem Statement

Users manually scan bank statements to:

- Calculate money received from someone
- Track payments sent
- Count transactions
- Verify financial activity

This process is slow, error-prone, and difficult for large statements.

The application solves this by automating extraction and filtering.

3. Product Goals (V1)

Primary Goals

- Upload PDF statement
- Extract text from PDF
- Search transactions by keyword/name
- Detect money direction:
 - Credit (Received)
 - Debit (Sent)
- Show total amount + count
- Show filtered transaction list

Non-Goals (Not in V1)

Do NOT build:

- Authentication
- MongoDB
- Charts
- Saved history
- AI categorization
- OCR
- Multi-file uploads
- Mobile app
- Analytics dashboards

Those belong to V2/V3.

4. Target Users

Primary Users

- Students
- Freelancers
- Small business owners
- Individuals tracking family/member transactions

Usage Example

A user uploads a statement and searches:

"Anil Vishwakarma"

The system returns:

- Total money received
 - Total money sent
 - Count of matching transactions
 - All matching rows
-

5. User Flow

Main Flow

Step 1 — Open Website

User lands on homepage.

Step 2 — Upload Statement

User uploads a single PDF bank statement.

Step 3 — Enter Search Keyword

Example:

- Anil
- Papa
- Vishwakarma

Step 4 — Select Filter Type

Options:

- Sent
- Received
- Both

Step 5 — Process

Backend extracts and filters transactions.

Step 6 — Results Displayed

Frontend shows:

- Total amount
- Transaction count
- Matching rows

6. Functional Requirements

6.1 File Upload

Requirements

- Accept only PDF files
- Max size: 10MB
- Single file upload only

Validation

Reject:

- Images
- DOC files
- ZIP files

Error Messages

Examples:

- "Only PDF files are allowed"
- "File size exceeds limit"

6.2 PDF Parsing

Backend Responsibilities

- Receive uploaded PDF
- Extract raw text
- Parse transaction rows

Technology

- `pdf-parse`

Output

Raw transaction text array.

6.3 Keyword Matching

Logic

System searches:

- Case insensitive
- Partial matches supported

Example:

Searching:

anil

Should match:

- Anil
 - ANIL
 - anil vishwakarma
-

6.4 Transaction Direction Detection

Rules

Credit Detection

Keywords:

- CR
- CREDIT
- RECEIVED

Debit Detection

Keywords:

- DR
 - DEBIT
 - SENT
-

6.5 Summary Calculation

Metrics

Calculate:

- Total amount
- Total transaction count

Example

Type	Value
Total Amount	₹45,000
Transactions	12

6.6 Transactions Table

Columns

Column	Description
Date	Transaction date
Description	Raw statement text
Type	Credit/D ebit
Amount	Transaction amount

7. UI Requirements

Design Style

- Minimal
 - Clean
 - Fast
 - Mobile responsive
-

Homepage Layout

Top Section

- App title
- Short description

Middle Section

Upload area

Search Section

- Keyword input
- Transaction type dropdown
- Analyze button

Results Section

- Summary cards
 - Transaction table
-

8. Technical Architecture

Frontend

Stack

- React.js
- Tailwind CSS
- Axios

Responsibilities

- File upload
 - Form handling
 - Display results
-

Backend

Stack

- Node.js
- Express.js

Responsibilities

- Receive PDF
 - Parse text
 - Filter transactions
 - Return JSON response
-

Packages

Package	Purpose
multer	File upload
pdf-parse	PDF extraction
cors	API communication
dotenv	Environment config

9. API Specification

Endpoint

POST /api/v1/analyzer/process

Request Type

multipart/form-data

Request Payload

Field	Type
file	PDF
keyword	String
type	sent/received/both

Response Example

```
{
  "success": true,
  "summary": {
    "totalAmount": 45000,
    "transactionCount": 12
  },
  "transactions": [
    {
      "date": "12-05-2026",
      "description": "UPI FROM ANIL",
      "type": "credit",
      "amount": 5000
    }
  ]
}
```

10. Error Handling

Scenario	Response
uploaded	400
Invalid PDF	400
Parsing failed	500
no results	200 with empty array

11. Performance Requirements

Expected Processing Time

- Under 5 seconds for standard PDFs

Concurrent Users

- V1 target: 10–20 concurrent users

12. Security Requirements

V1 Basic Security

- File type validation
 - File size limits
 - No permanent file storage
 - Delete temp uploads immediately
-

13. Future Scope (Reference Only)

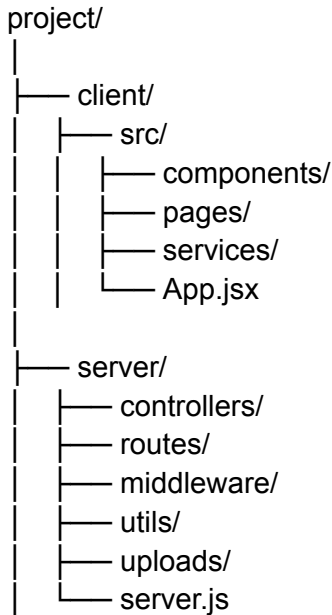
V2

- Daily/monthly summaries
- Date grouping
- Better parser engine

V3

- Authentication
 - MongoDB
 - Charts
 - Saved profiles
 - Historical analytics
-

14. Suggested Folder Structure



15. MVP Success Criteria

V1 is successful if:

- User uploads PDF successfully
 - Transactions are parsed correctly
 - Keyword filtering works reliably
 - Totals are accurate
 - UI remains simple and responsive
-

16. Critical Engineering Decision

Do not over-engineer V1.

Most beginners fail because they:

- Add auth too early
- Add DB too early
- Add dashboards too early

Your actual technical challenge is:

reliable PDF parsing + transaction normalization

Solve that first.

Everything else is secondary.